

hdc_core_axi_lite.sv — AXI4-Lite Wrapper and Co-Simulation Flow

This note documents the AXI4-Lite slave that exposes the verified end-to-end inference core (`hdc_core_top`) to the Zynq PS, and the co-simulation that drives the real register-map programming sequence over AXI and checks every prediction against the Python golden. For the protocol itself, see the companion `AXI4_Lite_Protocol_Study.pdf` .

1. Purpose

The PS (ARM cores) needs to: load the trained class prototypes and pruning mask once, then repeatedly hand the PL a quantized EMG window and read back the predicted gesture. `hdc_core_axi_lite.sv` provides that as a memory-mapped register block; software sees plain `Xil_Out32` / `Xil_In32` accesses.

2. Register map

Offset	Name	Access	Meaning
0x000	CTRL	W	bit0 <code>START</code> , bit1 <code>LOAD_PROTO</code> , bit2 <code>LOAD_MASK</code> , bit3 <code>CLR_DONE</code> (write-1-to-pulse)
0x004	STATUS	R	bit0 <code>BUSY</code> , bit1 <code>DONE</code> (sticky until <code>CLR_DONE</code>)
0x008	PROTO_IDX	RW	class index targeted by <code>LOAD_PROTO</code>
0x00C	RESULT	R	(<code>class_idx << 16</code>) <code>class_dist</code> ; bit31 = <code>DONE</code>
0x010	LEVELS0	RW	<code>levels_flat[31:0]</code>
0x014	LEVELS1	RW	<code>levels_flat[63:32]</code>
0x018	LEVELS2	RW	<code>levels_flat[79:64]</code> (low 16 bits used)
0x100 – 0x17C	STAGING	RW	32 words = one 1024-bit hypervector (word <code>w</code> = bits <code>[w*32 + : 32]</code>)

Programming sequence

```
for k in 0..N_CLASS-1:
    write STAGING[0..31] = prototype[k] words
    write PROTO_IDX = k
    write CTRL = LOAD_PROTO          # commits STAGING -> proto[k]
write STAGING[0..31] = mask words
write CTRL = LOAD_MASK              # commits STAGING -> mask
loop per window:
    write LEVELS0..2 = packed level grid
```

```

write CTRL = START
poll STATUS until DONE
read RESULT -> class_idx, class_dist
write CTRL = CLR_DONE

```

The **staging buffer** is the standard way to move data wider than the 32-bit bus: software fills the 1024-bit buffer with 32 ordinary word writes, then a single `LOAD_PROTO / LOAD_MASK` pulse commits the whole vector into the PL.

3. Internals

`hdc_core_axi_lite` holds the software-visible state (`staging_vec` , `proto_idx_reg` , `levels_reg` , `busy` / `done` , latched `result_idx` / `result_dist`) and instantiates one `hdc_core_top` . The control bits become one-cycle pulses to the core:

- `LOAD_PROTO` → `core.proto_we` (with `proto_idx_reg` / `staging_vec`)
- `LOAD_MASK` → `core.mask_we` (with `staging_vec`)
- `START` → `core.start` (with `levels_reg`); sets `BUSY`
- on `core.out_valid` → latch `class_idx` / `class_dist` , clear `BUSY` , set `DONE`

The AXI write/read handshakes follow the standard VALID/READY pattern (accept `AW + W` together, return `B` ; accept `AR` , return `R`) and honour `WSTRB` for byte-granular writes.

4. Co-simulation

Testbench — `tb/tb_core_axi_cosim.sv`

Contains a small AXI4-Lite **master model** (`axi_write` / `axi_read` tasks) that performs the handshakes exactly as a real interconnect would. It reads the same golden produced by `generate_vectors.py --core` (trained prototypes, pruning mask, level grids, expected labels), runs the full programming sequence above, and compares both `class_idx` and `class_dist` for every window. The item memories are loaded by the DUT's `item_mem` ROMs from the same `.mem` files.

Harness — `sim/run_core_axi_cosim.do`

```

vsim -c -do sim/run_core_axi_cosim.do      # NUM_CASES env var (default 200)

```

Regenerates golden → fresh `work` → compiles the full core + wrapper + TB → simulates the AXI-driven inference loop.

Flow

```

generate_vectors.py --core      tb_core_axi_cosim.sv (AXI master)
hdc_core_axi_lite.sv
-----
-----

```

```

trained protos / mask / levels -> $readmemh core_*.hex
classify (golden) -> expected    STAGING+CTRL: load protos/mask -> staging -> core
config                           LEVELS+START                        -> hdc_core_top
inference                        poll STATUS.DONE, read RESULT  <-  class_idx /
class_dist                       compare idx & dist              PASS ($finish) /
FAIL ($fatal)

```

5. Result

```

tb_core_axi_cosim: AXI4-Lite + hdc_core_top vs golden
  VECDIR = python_ref/vectors/cosim_core
  CASES  = 200    (D=1024, N_CLASS=8)
PASS: all 200 AXI-driven inferences match the Python golden.

```

200 / 200 inferences — driven entirely through the AXI4-Lite register map, including the staging-buffer prototype/mask loads — match the Python golden on ModelSim SE-64 10.6e. A matching bare-metal driver is in `sw/hdc_core_axi_example.c`.

This closes the loop from algorithm to bus interface: the PS can now configure and operate the HDC core through a clean register map. The remaining platform work is the high-throughput streaming path (`hdc_stream_wrapper.sv` : AXI4-Stream + DMA) and the on-board Zynq measurements.